JavaScript: Searching & Validating Data in Arrays

Revision guide · From basics to React · Concepts, shortcuts, mistakes, interview questions, and practice exercises · Print edition

Contents

1. Mental Model . . . 2
2. String Methods: Searching Inside a String . . . 2
3. Array Methods: Searching Across the Array . . . 3
4. Which Method Should I Use? . . . 5
5. Normalization: Making Search Forgiving . . . 5
6. Searching Multiple Fields . . . 6
7. Validation & Edge Cases . . . 6
8. Regular Expressions . . . 7
9. The Complete Search Function . . . 9
10. Testing the Search (QE Mindset) . . . 10
11. Beyond the Basics . . . 11
12. Search in React . . . 12
13. Performance . . . 14
14. Common Mistakes . . . 15
15. Cheat Sheet & Shortcuts . . . 15
16. Interview Questions . . . 16
17. Practice Exercises . . . 19
18. Summary . . . 19

Answer Key: Practice Exercises . . . 20

Sample data used throughout:

```
const contactsArr = [
  { name: 'John Smith', email: 'john.smith@gmail.com', phone: '415-792-6341' },
  { name: 'Kofi Mensah', email: 'kofi.mensah@yahoo.com', phone: '+233-24-597-1823' }
];
```

1. Mental Model

Searching an array of objects happens at **two levels**:

Level	What you check	Tools
Inside a string	Does `'John Smith'` contain `'smith'`?	`includes`, `startsWith`, `indexOf`, regex
Across the array	Which contacts match?	`filter`, `find`, `some`, `every`

The pipeline:

```
STEP            EXAMPLE
raw input       "  JOHN "
    ↓
normalize       "john"
    ↓
validate        not empty? is a string?
    ↓
loop items      contactsArr
    ↓
check fields    name / email / phone
    ↓
collect         filter → [ ...matches ]
```

TIP

Remember: the array method decides *what you get back;* the string method decides *what counts as a match*.

2. String Methods: Searching Inside a String

Method	Returns	Use when	ES version
`str.includes(t)`	`true` / `false`	Contains anywhere (most common)	ES6
`str.startsWith(t)`	`true` / `false`	"Begins with" / autocomplete	ES6
`str.endsWith(t)`	`true` / `false`	Suffix check, e.g. email domain	ES6
`str.indexOf(t)`	index or `-1`	You need the *position* (highlighting)	ES1
`str.lastIndexOf(t)`	index or `-1`	Last occurrence	ES1
`str.search(regex)`	index or `-1`	Position of a regex match	ES3
`regex.test(str)`	`true` / `false`	Pattern matching	ES3

```
const name = 'John Smith';

name.includes('Smith');        // true
name.includes('smith');        // false  ← case-sensitive!
name.startsWith('Jo');         // true
name.endsWith('th');           // true
name.indexOf('Smith');         // 5
name.indexOf('xyz');           // -1
name.includes('Smith', 6);     // false  ← optional 2nd arg = start position
/smith/i.test(name);           // true   ← "i" flag ignores case

'john.smith@gmail.com'.endsWith('@gmail.com');   // true
```

⚠ String gotchas

```
// 1. Everything is case-sensitive
'John'.includes('john');       // false

// 2. Empty string is "found" in EVERY string
'abc'.includes('');            // true  ← empty search matches everything!

// 3. indexOf returns 0 for a match at the start, and 0 is falsy
if ('John'.indexOf('Jo')) { }  // ✗ never runs, because indexOf returned 0
if ('John'.indexOf('Jo') !== -1) { }  // ✓
if ('John'.includes('Jo')) { }        // ✓ clearer
```

3. Array Methods: Searching Across the Array

All of these (except `includes` / `indexOf`) take a **callback** `(item, index, array) => boolean`.

Method	Returns	Stops early?	On empty array	Typical question
`find(cb)`	first item or `undefined`	✓	`undefined`	"Give me *the* contact named X"
`findIndex(cb)`	index or `-1`	✓	`-1`	"Where is it? (to update/delete)"
`findLast(cb)`	last item or `undefined`	✓	`undefined`	"Most recent matching entry" (ES2023)
`findLastIndex(cb)`	index or `-1`	✓	`-1`	Same, but index (ES2023)
`filter(cb)`	new array (maybe `[]`)	✗	`[]`	"Show all matches" (search box)
`some(cb)`	boolean	✓ on first `true`	`false`	"Does at least one match?"
`every(cb)`	boolean	✓ on first `false`	`true`	"Do all items pass?" (validation)
`includes(v)`	boolean	✓	`false`	Primitive arrays only
`indexOf(v)`	index or `-1`	✓	`-1`	Primitive arrays, need position

Examples

```
// find → first matching OBJECT
contactsArr.find(c => c.name.includes('Kofi'));
// { name: 'Kofi Mensah', email: 'kofi.mensah@yahoo.com', phone: '+233-24-597-1823' }

// findIndex → INDEX
contactsArr.findIndex(c => c.name.includes('Kofi'));   // 1

// filter → ARRAY of all matches
contactsArr.filter(c => c.email.includes('gmail'));    // [ { name: 'John Smith', ... } ]

// some → at least one?
```

```
contactsArr.some(c => c.email.endsWith('@yahoo.com')); // true

// every → all of them?
contactsArr.every(c => c.email.includes('@'));         // true
```

`find` vs `filter`

```
contactsArr.find(c => c.name === 'Nobody');    // undefined
contactsArr.filter(c => c.name === 'Nobody');  // []
```

	`find`	`filter`
Returns	one item	array
No match	`undefined` (falsy)	`[]` (**truthy!**)
Stops early	yes	no, checks everything
Use for	unique lookups (by id/email)	search results lists

`some` vs `every` on empty arrays (vacuous truth)

```
[].some(x => x > 0);    // false  → "is there at least one?" No.
[].every(x => x > 0);   // true   → "is there any that fails?" No, so true.
```

WATCH OUT
`every` on an empty array returns `true`. If "no data" should fail validation, check `arr.length > 0 && arr.every(...)`.

⚠ The `includes` trap with objects

```
contactsArr.includes({ name: 'John Smith', email: 'john.smith@gmail.com', phone: '415-792-6341'
    });
// false
```

Why: objects are compared by **reference** (same object in memory), not by content. A new object literal is always a new reference.

```
const john = contactsArr[0];
contactsArr.includes(john);                                  // true  (same reference)
contactsArr.some(c => c.email === 'john.smith@gmail.com'); // true  ✓ compare a field

// includes is perfect for primitives:
const emails = contactsArr.map(c => c.email);
emails.includes('john.smith@gmail.com');                  // true  ✓
```

`includes` vs `indexOf` (primitives)

```
[NaN].indexOf(NaN);    // -1    ← uses ===, and NaN !== NaN
[NaN].includes(NaN);   // true  ← uses SameValueZero
```

Prefer `includes` when you only need yes/no: it's more readable and handles `NaN`.

⚠ `find` returns a reference, not a copy

```
const found = contactsArr.find(c => c.name === 'John Smith');
found.phone = '000';            // ✗ this changes contactsArr[0] too!
```

`filter` returns a **new array**, but the objects inside are the **same objects** (shallow copy). In React, never mutate them; create new objects with spread: `{ ...found, phone: '000' }`.

4. Which Method Should I Use?

I need to...	Use
Show all matching contacts	`filter`
Get one contact by unique field	`find`
Update or delete an item in place	`findIndex`
Check "already exists?"	`some`
Check "all valid?"	`every`
Check a value in a primitive array	`includes`
Get a list of one field	`map` (then `includes`)
Check exact membership many times, fast	`Set.has` / `Map.get`

5. Normalization: Making Search Forgiving

Users type `"  JOHN "`, not `"John"`. **Normalize both sides** (the term *and* the data) before comparing.

```
const normalize = (str = '') =>
  String(str)
    .normalize('NFD')                  // split letters from accents: é → e + accent mark
    .replace(/[\u0300-\u036f]/g, '')    // remove the accent marks
    .trim()                            // remove outer spaces
    .toLowerCase();                    // ignore case

const digitsOnly = (str = '') => String(str).replace(/\D/g, '');   // \D = non-digit

normalize('  JOHN ');            // 'john'
normalize('José García');        // 'jose garcia'
digitsOnly('+233-24-597-1823');  // '233245971823'
digitsOnly('415 792');           // '415792'
```

Problem	Example	Fix
Case	`JOHN` vs `John`	`.toLowerCase()` both sides
Outer spaces	`"  john "`	`.trim()`
Phone format	`4157926341` vs `415-792-6341`	`.replace(/\D/g, '')` both sides
Accents	`jose` vs `José`	`.normalize('NFD').replace(/[\u0300-\u036f]/g, '')`
Non-string values	`null`, numbers	`String(value)` + default params

Shortcut: `localeCompare` for case/accent-insensitive equality

```
'José'.localeCompare('jose', undefined, { sensitivity: 'base' });   // 0 → equal
```

`sensitivity: 'base'` ignores case and accents. Useful for **exact** comparisons (not "contains").

6. Searching Multiple Fields

Option A: Explicit fields with `||` (recommended)

```
const term = 'kofi';

contactsArr.filter(c =>
  normalize(c.name).includes(term) ||
  normalize(c.email).includes(term)
);
```

Option B: Configurable field list

```
const SEARCH_FIELDS = ['name', 'email'];

contactsArr.filter(c =>
  SEARCH_FIELDS.some(field => normalize(c[field]).includes(term))
);
```

Adding a searchable field is now a one-word change.

Option C: Search every value

```
contactsArr.filter(c =>
  Object.values(c).some(value => normalize(value).includes(term))
);
```

Option	Pros	Cons
A: explicit `||`	Clear, full control	Repetitive with many fields
B: field list	Clean, easy to extend	Slightly more abstract
C: `Object.values`	Shortest	Searches hidden fields (ids, flags, nested objects become `"[object Object]"`)

7. Validation & Edge Cases

Validating the search term

```
function isValidTerm(term) {
  if (typeof term !== 'string') return false;   // null, undefined, numbers
  if (term.trim() === '') return false;          // empty / spaces only
  return true;
}
```

Decisions to make (and test)

Input / situation	Recommended behavior
`''` (empty)	Return **all** contacts (search box cleared)
`'  '` (spaces only)	Treat as empty
`null` / `undefined` / number	Don't crash; return all (or `[]`)
No match	Return `[]`; UI shows "No contacts found"
Contact missing a field	Use defaults so `.toLowerCase()` doesn't throw
Very long input	Optionally cap with `maxLength` on the input
Special characters (`+`, `.`, `()`)	Must not crash (watch out with regex)

Guarding against missing fields

```
// ✗ Crashes if a contact has no phone
contacts.filter(c => c.phone.includes(term));
    // TypeError: Cannot read properties of undefined

// ✓ Option 1: default values in destructuring
contacts.filter(({ phone = '' }) => phone.includes(term));

// ✓ Option 2: optional chaining + nullish coalescing
contacts.filter(c => (c.phone?.includes(term) ?? false));
```

Operator	Meaning
`a?.b`	If `a` is `null`/`undefined`, return `undefined` instead of throwing
`a ?? b`	Use `b` only if `a` is `null`/`undefined` (unlike `||`, keeps `0` and `''`)

Validating the data itself

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

const allEmailsValid = contactsArr.length > 0 &&
  contactsArr.every(c => emailPattern.test(c.email));        // true

const hasDuplicateEmails =
  new Set(contactsArr.map(c => c.email.toLowerCase())).size !== contactsArr.length;   // false
```

8. Regular Expressions

Basics

```
/smith/i.test('John Smith');           // true   → boolean
'John Smith'.match(/smith/i);          // ['Smith', index: 5, ...] → match details
'John Smith'.search(/smith/i);         // 5      → position
```

Flag	Meaning
`i`	Ignore case
`g`	Global (all matches); **makes the regex stateful**
`m`	Multiline (^/$ match per line)
`u`	Unicode-aware

Token	Meaning
`\d` / `\D`	Digit / non-digit
`\w` / `\W`	Word char `[A-Za-z0-9_]` / non-word
`\s`	Whitespace
`\b`	Word boundary
`^` / `$`	Start / end of string

Building a regex from user input: ESCAPE IT

```
new RegExp('+233');
//  SyntaxError: Invalid regular expression: /+233/: Nothing to repeat
```

`+` is a regex quantifier. Users type `+`, `.`, `(`, `?` all the time, so always escape:

```
const escapeRegExp = (str) => str.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

new RegExp(escapeRegExp('+233'), 'i');   // ✓ /\+233/i
```

> **NOTE**
> In a template literal, write `\\b` to get `\b` in the regex: `new RegExp(`\\b${term}`)`.

Word-start matching

```
const matchesWordStart = (text, term) =>
  new RegExp(`\\b${escapeRegExp(term)}`, 'i').test(text);

matchesWordStart('John Smith', 'smi');    // true  ✓ "Smi" starts a word
matchesWordStart('John Smith', 'mith');   // false ✗ middle of a word
```

> **WATCH OUT**
> `\b` only works when the term starts with a word character. For `'+233'` there's no word boundary before `+`, so use `includes` for phones.

⚠ The `g` flag + `test()` trap

```
const re = /john/gi;
re.test('John');   // true
re.test('John');   // false
```

Why: with `g`, the regex remembers `lastIndex` (where the last match ended) and starts the next search from there. Using one `g` regex inside `filter` gives **alternating wrong results**.

Fix: don't use `g` with `test()`, or create the regex inside the callback, or reset `re.lastIndex = 0`.

> **NOTE**
> **Rule of thumb:** use `includes` by default. Reach for regex only for patterns (word boundaries, validation formats).

9. The Complete Search Function

```
// ---------- Helpers ----------
const normalize = (str = '') =>
  String(str).normalize('NFD').replace(/[\u0300-\u036f]/g, '').trim().toLowerCase();

const digitsOnly = (str = '') => String(str).replace(/\D/g, '');

// Only digits, spaces, +, -, (, ), . → treat as a phone search
const isPhoneLike = (str) => /^[+\d\s().-]+$/.test(str);

// ---------- Search ----------
function searchContacts(contacts, rawTerm) {
  // 1. Validate inputs
  if (!Array.isArray(contacts)) return [];
  if (typeof rawTerm !== 'string') return contacts;

  // 2. Normalize the term
  const term = normalize(rawTerm);

  // 3. Empty search → show everything
  if (term === '') return contacts;

  // 4. Filter
  return contacts.filter(({ name = '', email = '', phone = '' }) => {
    if (isPhoneLike(term)) {
      const termDigits = digitsOnly(term);
      return termDigits !== '' && digitsOnly(phone).includes(termDigits);
    }
    return normalize(name).includes(term) || normalize(email).includes(term);
  });
}

// ---------- Validation helper ----------
const hasMatch = (contacts, term) => searchContacts(contacts, term).length > 0;
```

Step by step:

Step	Why
`Array.isArray` check	Bad data from an API shouldn't crash the UI
`typeof rawTerm` check	`null`/`undefined` from an uncontrolled input
`normalize(rawTerm)`	Case, spaces, accents
Empty → return all	`''.includes` matches everything anyway; make it explicit
Default params `name = ''`	Missing fields don't throw
`isPhoneLike` branch	Phones compared digit-to-digit, so formatting doesn't matter
`termDigits !== ''`	Guards against a term like `'+'` matching every phone

10. Testing the Search (QE Mindset)

Test cases

#	Input	Expected count	What it proves
1	`'john'`	1	Basic name match
2	`'  KOFI '`	1	Trim + case-insensitive
3	`'gmail'`	1	Email field searched
4	`'4157926341'`	1	Phone without dashes
5	`'415-792'`	1	Partial phone with dashes
6	`'+233'`	1	Special char doesn't crash
7	`''`	2	Empty → all
8	`'   '`	2	Spaces only → all
9	`'zzz'`	0	No match → []
10	`null`	2	Invalid type doesn't crash
11	contact with no phone, term `'123'`	0	Missing field handled
12	`'jose garcia'` vs `'José García'`	1	Accent-insensitive

Quick test runner (paste into your editor)

```
const tests = [
  ['john', 1], ['  KOFI ', 1], ['gmail', 1], ['4157926341', 1], ['415-792', 1],
  ['+233', 1], ['', 2], ['   ', 2], ['zzz', 0], [null, 2],
];

tests.forEach(([input, expected]) => {
  const actual = searchContacts(contactsArr, input).length;
  console.log(actual === expected ? '✓' : '✗', JSON.stringify(input), '→', actual);
});
```

Test design techniques

Technique	Applied to search
Equivalence partitioning	Name terms, email terms, phone terms, empty, invalid type
Boundary values	1-character term, full exact value, value + 1 extra char
Negative testing	`null`, numbers, special chars, emoji, very long strings
Data variations	Missing fields, accents, duplicate names, empty array

11. Beyond the Basics

Multi-word search ("john gmail")

Every word must match *somewhere* in the contact:

```
function multiWordSearch(contacts, raw) {
  const words = normalize(raw).split(/\s+/).filter(Boolean);
  if (words.length === 0) return contacts;

  return contacts.filter(c => {
    const haystack = normalize(`${c.name} ${c.email}`);
    return words.every(word => haystack.includes(word));
  });
}

multiWordSearch(contactsArr, 'john gmail');   // [John Smith]
multiWordSearch(contactsArr, 'john yahoo');   // []
```

NOTE

`filter(Boolean)` removes empty strings. **Haystack** = all searchable text joined into one string.

Relevance ranking (best matches first)

```
const score = (contact, term) => {
  const n = normalize(contact.name);
  if (n === term) return 3;                                    // exact
  if (n.startsWith(term)) return 2;                            // starts with
  if (n.split(/\s+/).some(word => word.startsWith(term))) return 1; // a word starts with
  return 0;                                                    // contains somewhere
};

const ranked = [...results].sort((a, b) => score(b, term) - score(a, term));
// or ES2023: results.toSorted(...)
```

NOTE

`sort()` **mutates** the array, so copy first with `[...results]` (important for React state) or use `toSorted()`.

Highlighting the match

```
function splitForHighlight(text, term) {
  const i = text.toLowerCase().indexOf(term.toLowerCase());
  if (!term || i === -1) return { before: text, match: '', after: '' };
  return {
    before: text.slice(0, i),
    match: text.slice(i, i + term.length),
    after: text.slice(i + term.length),
  };
}

splitForHighlight('John Smith', 'smi');
// { before: 'John ', match: 'Smi', after: 'th' }
```

WATCH OUT

If you strip accents for matching, positions can shift. Highlight using the same normalization as matching, or keep highlighting case-only.

Fuzzy search (typos)

"Jhon" → "John" needs fuzzy matching (edit distance). Don't build this by hand for real projects; libraries like **Fuse.js** exist. Know the term for interviews.

12. Search in React

Core pattern: derived state

```
import { useState } from 'react';

function ContactSearch({ contacts }) {
  const [query, setQuery] = useState('');

  // ✓ Derived: computed from state on every render
  const results = searchContacts(contacts, query);

  return (
    <div>
      <input
        type="search"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search name, email or phone"
        aria-label="Search contacts"
      />

      <p aria-live="polite">{results.length} result(s)</p>

      {results.length === 0 ? (
        <p>No contacts found for “{query}”</p>
      ) : (
        <ul>
          {results.map((c) => (
            <li key={c.email}>
              {c.name} · {c.email} · {c.phone}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

✗ Anti-pattern: storing results in state

```
const [query, setQuery] = useState('');
const [results, setResults] = useState(contacts);

useEffect(() => {
  setResults(searchContacts(contacts, query));   // ✗ extra render, can go stale
}, [query, contacts]);
```

Rule: if a value can be **calculated** from existing props/state, don't put it in state.

Optimizations: when to use what

Tool	Use when	Example
Plain derived value	Default; small/medium lists	`const results = search(...)`
`useMemo`	Expensive filtering on **large** lists	`useMemo(() => search(c, q), [c, q])`
`useDeferredValue`	Typing feels laggy because rendering results is slow	`const dq = useDeferredValue(q)`
Debounce	Each search triggers an **API call**	Wait 300 ms after typing stops

```
// useMemo
const results = useMemo(() => searchContacts(contacts, query), [contacts, query]);

// useDeferredValue (React 18+): input stays responsive, list updates slightly later
const deferredQuery = useDeferredValue(query);
const results = useMemo(
  () => searchContacts(contacts, deferredQuery),
  [contacts, deferredQuery]
);
```

Debounce hook (builds on `setTimeout` + cleanup)

```
function useDebouncedValue(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);   // cleanup: cancel if value changes before delay
  }, [value, delay]);

  return debounced;
}

// Usage
const debouncedQuery = useDebouncedValue(query, 300);
useEffect(() => {
  if (debouncedQuery) fetchContacts(debouncedQuery);
}, [debouncedQuery]);
```

Highlight component

```
function Highlight({ text, term }) {
  const { before, match, after } = splitForHighlight(text, term.trim());
  return (
    <>
      {before}
      {match && <mark>{match}</mark>}
      {after}
    </>
  );
}

// <li key={c.email}><Highlight text={c.name} term={query} /></li>
```

React checklist

- Controlled input: `value` + `onChange`
- Results are **derived**, not stored
- Stable, unique `key` (email/id), **not** the array index when the list is filtered

- Never mutate contacts; use spread for updates
- Empty state + result count for UX and accessibility (`aria-live`)

13. Performance

Operation	Complexity	Notes
`filter` / `find` / `some`	O(n)	Checks items one by one
`arr.includes(v)`	O(n)	Linear scan
`set.has(v)`	O(1) average	Hash lookup
`map.get(k)`	O(1) average	Hash lookup, returns value

Set / Map for repeated exact lookups

```
// Build once: O(n)
const emailSet = new Set(contactsArr.map(c => c.email.toLowerCase()));
const byEmail  = new Map(contactsArr.map(c => [c.email.toLowerCase(), c]));

// Lookup many times: O(1)
emailSet.has('john.smith@gmail.com');       // true
byEmail.get('kofi.mensah@yahoo.com');       // { name: 'Kofi Mensah', ... }
```

Need	Use
Partial "contains" search	`filter` (Set/Map can't help)
One-off exact check	`some` / `find`
Many exact checks on big data	`Set` / `Map`

Other tips

- Normalize data **once** (e.g. precompute `searchText` per contact) instead of inside every keystroke's filter, for large lists.
- Prefer `some`/`find` over `filter(...).length > 0` when you just need yes/no: they stop early.

14. Common Mistakes

✗ Mistake	Why it's wrong	✓ Fix
`arr.includes({...})`	Reference comparison	`arr.some(c => c.id === x)`
`if (arr.filter(...))`	`[]` is truthy	`if (arr.some(...))` or check `.length`
`if (str.indexOf(t))`	Match at index 0 is falsy	`str.includes(t)`
Normalizing only the term	`'john'` vs `'John'` still fails	Normalize **both** sides
`c.phone.includes(t)` with missing phone	TypeError	Default params / `?.`
`new RegExp(userInput)`	Crashes on `+`, `(`, etc.	`escapeRegExp` first
Reusing a `/g` regex with `test()`	Stateful `lastIndex`	Drop `g`
`filter(...)[0]` for one item	Scans whole array	`find(...)`
`results.sort()` on state	Mutates array	`[...results].sort()` / `toSorted()`
Storing filtered results in state	Duplicate state, stale bugs	Derive during render
key={index} in filtered list	Keys shift as list changes	Use unique id/email
Forgetting empty term case	`''.includes` matches all, maybe unintentionally	Handle explicitly

15. Cheat Sheet & Shortcuts

```
// Contains (case-insensitive)
str.toLowerCase().includes(term.toLowerCase())

// Exists?
arr.some(c => c.email === x)

// Get one
arr.find(c => c.id === id) ?? null

// Get all
arr.filter(c => c.name.toLowerCase().includes(t))

// Remove by id (immutable)
arr.filter(c => c.id !== id)

// Update by id (immutable)
arr.map(c => c.id === id ? { ...c, phone: newPhone } : c)

// All valid?
arr.length > 0 && arr.every(c => c.email.includes('@'))

// Duplicates?
new Set(arr.map(c => c.email)).size !== arr.length

// Unique values
[...new Set(arr.map(c => c.country))]

// Count matches
arr.filter(fn).length

// Digits only
str.replace(/\D/g, '')
```

```
// Escape for regex
str.replace(/[.*+?^${}()|[\]\\]/g, '\\$&')

// Case + accent-insensitive equality
a.localeCompare(b, undefined, { sensitivity: 'base' }) === 0

// Remove empty strings
words.filter(Boolean)

// Safe field access
c.phone?.includes(t) ?? false
```

Memory hooks

- **find** = *one* · **filter** = *many* · **some** = *any?* · **every** = *all?*
- `[]` **is truthy**, `0` **is falsy**, `''` **is in every string**
- **Objects → reference**, **primitives → value**
- **Normalize → Validate → Filter → Derive**

16. Interview Questions

> **NOTE**
> Cover each answer with a sheet of paper. Answer out loud first, then check.

Beginner

Q1. What's the difference between `find` and `filter`?

> **ANSWER**
> `find` returns the **first** matching element (or `undefined`) and stops searching once found. `filter` returns a **new array** of **all** matches (or `[]`) and always checks every element. Use `find` for unique lookups (by id), `filter` for search results.

Q2. What's the difference between `some` and `every`?

> **ANSWER**
> `some` returns `true` if **at least one** element passes the test (stops at the first `true`). `every` returns `true` only if **all** pass (stops at the first `false`). `some` → "does any exist?", `every` → "are all valid?".

Q3. Why does `contactsArr.includes({ name: 'John Smith', ... })` return `false`?

> **ANSWER**
> `includes` compares using SameValueZero, which for objects means **reference equality**. A new object literal creates a new reference, even with identical content. Use `some(c => c.email === x)` to compare by a field.

Q4. How do you make a search case-insensitive?

> **ANSWER**
> Lowercase **both** sides: `str.toLowerCase().includes(term.toLowerCase())`. Alternatives: a regex with the `i` flag (escape user input first), or `localeCompare` with `{ sensitivity: 'base' }` for exact equality.

Q5. What does `'abc'.includes('')` return, and why does it matter?

> **ANSWER**
> `true`. The empty string is considered present in every string. So an empty search term matches every item; handle the empty case explicitly (usually "show all").

Q6. Why is `if (str.indexOf('Jo'))` a bug?

> **ANSWER**
> If the match is at the start, `indexOf` returns `0`, which is falsy, so the block doesn't run even though there's a match. Compare with `!== -1`, or use `includes`.

Intermediate

Q7. What's the difference between `includes` and `indexOf` on arrays?

> **ANSWER**
> `includes` returns a boolean and uses SameValueZero, so `[NaN].includes(NaN)` is `true`. `indexOf` returns a position and uses `===`, so `[NaN].indexOf(NaN)` is `-1`. Use `includes` for yes/no checks, `indexOf` when you need the position.

Q8. What do `[].some(fn)` and `[].every(fn)` return?

> **ANSWER**
> `some` → `false` (there isn't at least one). `every` → `true` (vacuous truth: nothing fails). If an empty array should fail validation, add `arr.length > 0 &&`.

Q9. Does `filter` mutate the original array? Does `find` return a copy?

> **ANSWER**
> `filter` doesn't mutate; it returns a new array. But it's a **shallow** copy: the objects inside are the same references. `find` returns the actual object, so mutating it mutates the original array. In React, always create new objects (`{ ...obj, field }`).

Q10. How do you search across multiple fields?

> **ANSWER**
> Combine checks with `||` in the callback, or keep a list of fields and use `fields.some(f => normalize(c[f]).includes(term))`. `Object.values(c).some(...)` also works but searches every field, including ones you don't want searchable.

Q11. How do you safely handle objects with missing fields?

> **ANSWER**
> Default values in destructuring (`({ phone = '' }) => ...`), or optional chaining with nullish coalescing (`c.phone?.includes(t) ?? false`). Otherwise `undefined.includes` throws a TypeError.

Q12. What's wrong with `new RegExp(userInput)`?

> **ANSWER**
> User input may contain regex special characters (`+ . * ? ( ) [ ]`), which either throw (`'+233'` → "Nothing to repeat") or change the meaning. Escape with `str.replace(/[.*+?^${}()|[\]\\]/g, '\\$&')` first.

Q13. Why does a regex with the `g` flag give alternating results with `test()`?

> **ANSWER**
> A global regex is stateful: it stores `lastIndex` after a match and starts the next `test()` from there. The second call on the same string starts past the match and fails, which resets `lastIndex` to 0. Don't use `g` with `test()` in a loop.

Advanced / React

Q14. Should filtered search results be stored in React state?

> **ANSWER**
> No. They're **derived** from `query` and `contacts`, so compute them during render. Storing them in state duplicates data, adds an extra render via `useEffect`, and can go stale if one source changes without the other.

Q15. When would you use `useMemo`, `useDeferredValue`, or debounce for search?

> **ANSWER**
> - `useMemo`: filtering is genuinely expensive (large lists); skips recalculation when inputs don't change.
> - `useDeferredValue`: rendering results is slow and typing lags; React keeps the input responsive and updates the list with lower priority.
> - Debounce: every search hits an **API**; wait until the user pauses typing to avoid flooding the server.

Q16. Why shouldn't you use the array index as `key` in a filtered list?

> **ANSWER**
> When the list is filtered, items shift positions, so index-based keys point to different items. React may reuse the wrong DOM/state (e.g. inputs keep wrong values). Use a stable unique id like email.

Q17. You need to check thousands of emails against a list of 50,000 users. How?

> **ANSWER**
> `some`/`includes` per check is O(n), so total O(n × m). Build a `Set` of normalized emails once (O(n)), then each `has()` is O(1) on average. Total ≈ O(n + m).

Q18. How would you implement accent-insensitive search?

> **ANSWER**
> `str.normalize('NFD')` splits accented characters into base letter + combining mark; then `.replace(/[\u0300-\u036f]/g, '')` removes the marks. Apply to both term and data. For equality only, `localeCompare(b, undefined, { sensitivity: 'base' })`.

Q19. How would you implement a multi-word search like "john gmail"?

> **ANSWER**
> Split the normalized term into words (`split(/\s+/).filter(Boolean)`), build a haystack string from the searchable fields, and keep contacts where `words.every(w => haystack.includes(w))`.

Q20. How would you test a search feature?

> **ANSWER**
> Equivalence classes (name/email/phone terms, empty, invalid type), boundary values (1 char, exact full value), negative tests (special chars, `null`, very long input), data variations (missing fields, accents, duplicates, empty list), and UI checks (empty state message, result count, debounce timing, accessibility of the input and live region).

Live-coding prompts

1. Write `isEmailTaken(contacts, email)`, case-insensitive.
2. Write `hasDuplicateEmails(contacts)` in one line.
3. Remove a contact by email **without mutating** the array.
4. Update a contact's phone **immutably**.

5. Return contacts grouped by email domain (`gmail.com`, `yahoo.com`).

17. Practice Exercises

> **NOTE**
> Do these in your editor. Solutions are in the Answer Key at the end, so you can try first without peeking.

E1. Write `findByEmail(contacts, email)` → contact or `null`, case-insensitive, ignores outer spaces.

→ *Solution: see the Answer Key, E1.*

E2. Write `isEmailTaken(contacts, email)` with `some`. Then with a `Set`.

→ *Solution: see the Answer Key, E2.*

E3. Match names only at the **start of a word** (`smi` ✓, `mith` ✗).

→ *Solution: see the Answer Key, E3.*

E4. Add a contact with **no** `phone` and confirm `searchContacts` doesn't crash for `'123'` and `'ann'`.

→ *Solution: see the Answer Key, E4.*

E5. Sort results so exact > starts-with > word-starts-with > contains.

→ *Solution: see the Answer Key, E5.*

E6. Detect duplicate emails (case-insensitive) in one line.

→ *Solution: see the Answer Key, E6.*

E7. Group contacts by email domain.

→ *Solution: see the Answer Key, E7.*

E8 (React). Build `ContactSearch` with: controlled input, derived results, result count, empty state, and `<Highlight>` on names.

→ *Hints: see the Answer Key, E8.*

18. Summary

Concept	One-liner
Two levels	String method = what matches; array method = what you get back
`find` / `filter`	One item vs. array of all
`some` / `every`	Any? vs. all? (`[].every` is `true`)
`includes` on objects	Reference comparison; use `some` with a field
Normalize	trim + lowercase (+ accents, + digits for phones), both sides
Validate	Empty → all, invalid type → no crash, missing fields → defaults
Regex	Escape user input; no `g` with `test()`
React	Derive results, don't store; `useMemo` / `useDeferredValue` / debounce only when needed
Performance	`filter` is O(n); `Set.has` is O(1) for exact lookups

Mantra

Normalize → Validate → Filter (||) → Derive in React

Answer Key: Practice Exercises

> **NOTE**
> Try each exercise in section 17 on your own before reading these.

E1 · Solution

```
const findByEmail = (contacts, email) =>
  contacts.find(c => c.email.toLowerCase() === email.trim().toLowerCase()) ?? null;

findByEmail(contactsArr, ' John.Smith@Gmail.com ');   // { name: 'John Smith', ... }
findByEmail(contactsArr, 'x');                         // null
```

E2 · Solution

```
// some: O(n) per check
const isEmailTaken = (contacts, email) =>
  contacts.some(c => c.email.toLowerCase() === email.trim().toLowerCase());

// Set: build once, O(1) per check
const takenEmails = new Set(contactsArr.map(c => c.email.toLowerCase()));
const isEmailTakenFast = (email) => takenEmails.has(email.trim().toLowerCase());
```

E3 · Solution

```
const escapeRegExp = (str) => str.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

const matchesWordStart = (text, term) =>
  new RegExp(`\\b${escapeRegExp(term)}`, 'i').test(text);

contactsArr.filter(c => matchesWordStart(c.name, 'smi'));    // [John Smith]
contactsArr.filter(c => matchesWordStart(c.name, 'mith'));   // []
```

E4 · Solution

```
const withMissing = [...contactsArr, { name: 'Ann Lee', email: 'ann@lee.com' }];
searchContacts(withMissing, '123').length;   // 0, no crash (phone defaults to '')
searchContacts(withMissing, 'ann').length;   // 1
```

E5 · Solution

```
const score = (c, t) => {
  const n = normalize(c.name);
  if (n === t) return 3;
  if (n.startsWith(t)) return 2;
  if (n.split(/\s+/).some(w => w.startsWith(t))) return 1;
  return 0;
};

const people = [{ name: 'Ann Johnson' }, { name: 'John Smith' }, { name: 'Bjohn X' }];
[...people].sort((a, b) => score(b, 'john') - score(a, 'john')).map(p => p.name);
// ['John Smith', 'Ann Johnson', 'Bjohn X']
```

E6 · Solution

```
const hasDuplicateEmails = (contacts) =>
  new Set(contacts.map(c => c.email.toLowerCase())).size !== contacts.length;

hasDuplicateEmails(contactsArr);                                          // false
hasDuplicateEmails([...contactsArr, { email: 'JOHN.SMITH@gmail.com' }]);  // true
```

E7 · Solution

```
const byDomain = contactsArr.reduce((groups, c) => {
  const domain = c.email.split('@')[1];
  (groups[domain] ??= []).push(c);
  return groups;
}, {});
// { 'gmail.com': [John], 'yahoo.com': [Kofi] }

// Modern alternative (ES2024): Object.groupBy(contactsArr, c => c.email.split('@')[1])
```

E8 · Hints

- `const [query, setQuery] = useState('')`
- `const results = searchContacts(contacts, query)`
- `key={c.email}`
- Reuse `splitForHighlight` from section 11 inside `<Highlight>`